

MAZE SOLVING VISUALIZATION USING STACK
EIGHTH-WEEK ASSIGNMENT FOR THE DATA STRUCTURE



Lecturer :

Andi Iwan Nurhidayat, S.Kom., M.T.

Presented by :

Aisyah Currents

25091397108 / 2025I

INFORMATIC MANAGEMENT MAJOR
FACULTY OF VOCATION
UNIVERSITAS NEGERI SURABAYA
YEAR 2026

1. Introduction

This project aims to create a program that can find a path in a maze from the starting point (.) to the exit (E). The program uses a stack data structure and the Depth First Search (DFS) algorithm. Overall, this project demonstrates how data structures and algorithms can be combined to solve real-world problems such as maze navigation in a logical and efficient way.

2. Objective

The objectives of this project are:

- a. To understand how a stack works (LIFO concept)
- b. To implement Depth First Search (DFS) in Python
- c. To visualize the pathfinding process using a GUI
- d. To find a path from the start point to the exit in a maze

3. Theoretical Background

a. Stack

A stack is a data structure that works based on the Last In First Out (LIFO) principle. This means the last element added (pushed) into the stack will be the first one to be removed (popped). You can imagine it like a stack of plates, when you add a new plate, it goes on top, and when you take one, you remove the top plate first. Stacks are commonly used in programming for tasks like undo operations, expression evaluation, and function calls.

b. Depth First Search (DFS)

Depth First Search (DFS) is an algorithm used to explore data structures like graphs or trees. It works by going as deep as possible into one path before backtracking and trying another path. DFS typically uses a stack (either explicitly or through recursion) to keep track of the nodes that need to be visited next. This approach helps in exploring all possible paths systematically.

c. Maze Solving

Maze solving is a problem where the goal is to find a path from a starting point to a destination or exit. Algorithms like DFS can be used to solve mazes by exploring all possible directions step by step. The algorithm keeps moving forward until it hits a dead end, then backtracks to try a different path. This process continues until the exit is found or all possible paths have been explored.

4. Algorithm Steps

The steps of the algorithm are:

1. Start from the starting point (.)
2. Push the start position into the stack
3. Pop the top position from the stack
4. Check if it is the exit (E)
5. If not, explore its neighbors (up, down, left, right)
6. Push valid positions into the stack
7. Mark visited positions
8. Repeat until the exit is found

5. Implementation

The program is implemented using Python and the pygame library.
The maze is represented as a 2D list.

Each cell can be:

- Wall (#)
- Path and Start (.)
- End (E)

The DFS algorithm uses a stack to explore the maze.

The Source Code:

```
1. import pygame
2.
3. pygame.init()
4.
5. # Window
6. WIDTH, HEIGHT = 600, 600
7. WIN = pygame.display.set_mode((WIDTH, HEIGHT))
8. pygame.display.set_caption("Maze BFS Shortest Path")
9.
10. # Colors
11. BG = (10, 30, 60)
12. WALL = (20, 40, 80)
13. PATH = (240, 225, 180)
14. VISITED = (150, 230, 200) # Cyan - being explored
15. FINAL = (50, 255, 100) # Bright green - solution path
16. DEAD_END = (255, 150, 50) # Bright orange - wrong path
17. WHITE_BLOCK = (230, 230, 230)
18.
19. START_COLOR = (0, 200, 150)
20. END_COLOR = (255, 170, 0)
21.
22. GRID = (30, 50, 90)
23.
24. # Maze (adjusted like image)
25. maze = [
26.     list("#####"),
27.     list("#...#...#...#"),
28.     list("###.#.#.###.#"),
29.     list("#....#...#.#"),
30.     list("#.#####.#...#"),
31.     list("#.#....#.#####"),
32.     list("#.#.###.#....##"),
33.     list("#...#.#.#####"),
34.     list("###.#.#....#.#"),
35.     list("#.#.#.#####.#"),
36.     list("#.#....#...#.#"),
37.     list("#.#####.####.#"),
38.     list("#....#.....#"),
39.     list("#####.#####E"),
40.     list("#####")
41. ]
```

```

42.
43. ROWS = len(maze)
44. COLS = len(maze[0])
45. CELL = WIDTH // COLS
46.
47. # Find start (first dot '.')
48. start = None
49. for i in range(ROWS):
50.     for j in range(COLS):
51.         if maze[i][j] == '.':
52.             start = (i, j)
53.             break
54.     if start:
55.         break
56.
57. # Draw
58. def draw():
59.     WIN.fill(BG)
60.
61.     for i in range(ROWS):
62.         for j in range(COLS):
63.             x, y = j * CELL, i * CELL
64.
65.             if maze[i][j] == '#':
66.                 color = WALL
67.             elif maze[i][j] == 'E':
68.                 color = END_COLOR
69.             elif maze[i][j] == 'x':
70.                 color = VISITED
71.             elif maze[i][j] == 'o':
72.                 color = FINAL
73.             elif maze[i][j] == 'v':
74.                 color = DEAD_END
75.             else:
76.                 color = PATH
77.
78.             pygame.draw.rect(WIN, color, (x, y, CELL, CELL))
79.             pygame.draw.rect(WIN, GRID, (x, y, CELL, CELL), 1)
80.
81.             # Draw symbols
82.             if maze[i][j] == 'x':
83.                 pygame.draw.circle(WIN, (100, 255, 150), (x+CELL//2, y+
CELL//2), 5)
84.             elif maze[i][j] == 'o':
85.                 pygame.draw.circle(WIN, (0, 255, 50), (x+CELL//2, y+CE
LL//2), 5)
86.             elif maze[i][j] == 'v':
87.                 pygame.draw.circle(WIN, (255, 100, 0), (x+CELL//2, y+CE
LL//2), 5)
88.             elif (i, j) == start:

```

```

89.             pygame.draw.circle(WIN, START_COLOR, (x+CELL//2, y+CELL
//2), 6)
90.
91.     pygame.display.update()
92.
93. # BFS + find shortest path
94. def bfs():
95.     from collections import deque
96.     queue = deque([(start, [start])])
97.     visited = set([start])
98.
99.     directions = [(-1,0),(1,0),(0,-1),(0,1)]
100.
101.     running = True
102.     shortest_path = None
103.
104.     while queue and running:
105.         for event in pygame.event.get():
106.             if event.type == pygame.QUIT:
107.                 running = False
108.
109.                 (x, y), path = queue.popleft()
110.
111.                 if maze[x][y] == 'E':
112.                     shortest_path = path
113.                     print("Exit Found! Shortest path length:", len(path))
114.                     break
115.
116.                 for dx, dy in directions:
117.                     nx, ny = x + dx, y + dy
118.
119.                     if 0 <= nx < ROWS and 0 <= ny < COLS:
120.                         if (nx, ny) not in visited and maze[nx][ny] in ['
.', 'E']:
121.                             visited.add((nx, ny))
122.                             queue.append(((nx, ny), path + [(nx, ny)]))
123.
124.                 if shortest_path:
125.                     # draw shortest path with green color
126.                     for px, py in shortest_path:
127.                         if (px, py) != start:
128.                             maze[px][py] = 'o'
129.                     draw()
130.                     pygame.time.delay(100)
131.
132.                     print("Animation completed!")
133.
134.     # Main
135.     def main():
136.         running = True
137.         started = False

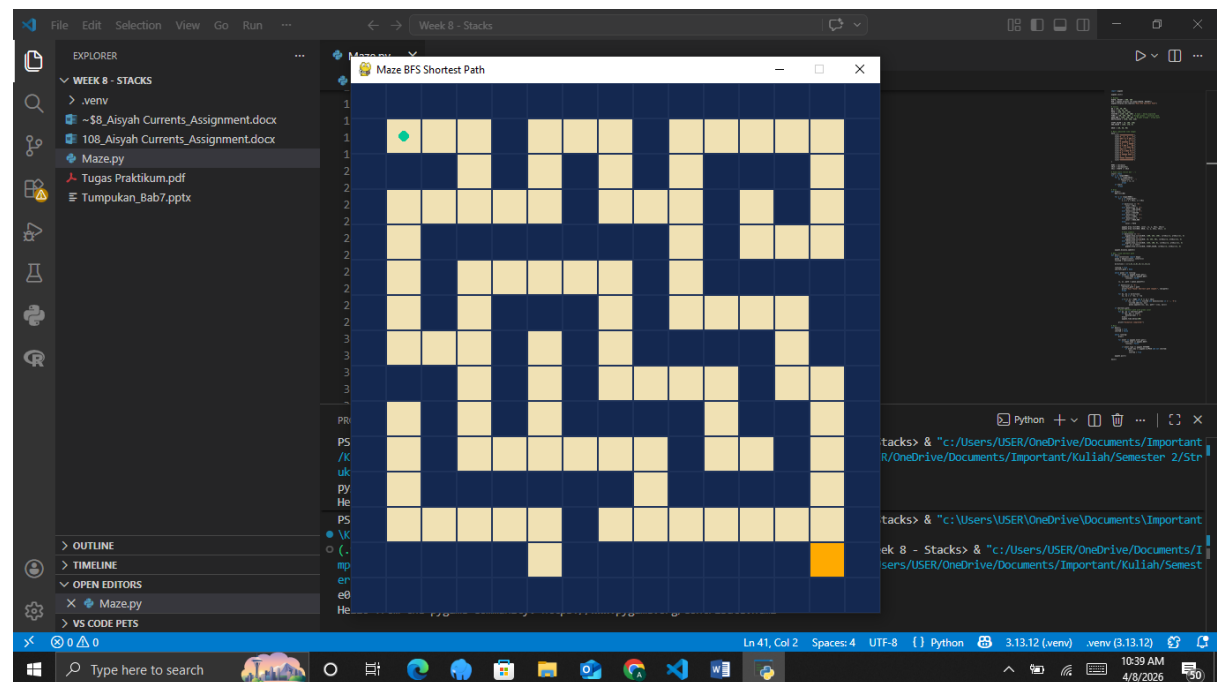
```

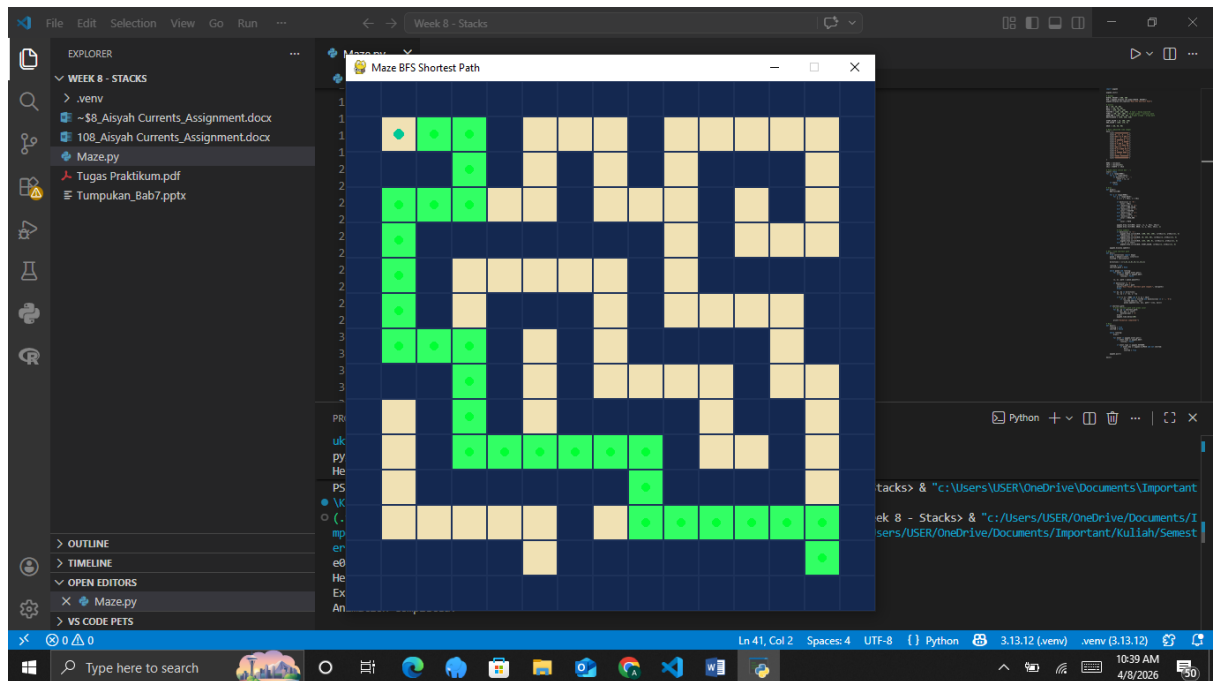
```

138.
139.     while running:
140.         draw()
141.
142.         for event in pygame.event.get():
143.             if event.type == pygame.QUIT:
144.                 running = False
145.
146.             if event.type == pygame.KEYDOWN:
147.                 if event.key == pygame.K_SPACE and not started:
148.                     bfs()
149.                     started = True
150.
151.         pygame.quit()
152.
153.     main()

```

6. Results





The Explanation:

This program uses the Breadth First Search (BFS) algorithm to find the shortest path in a maze from the starting point to the exit. The maze is displayed using the pygame library, where different colors represent walls, paths, visited cells, and the final solution. The starting point is automatically determined from the first available path ('.'), and the goal is marked as 'E'. The BFS algorithm works by exploring all neighboring positions level by level using a queue, ensuring that the shortest path is found. During the process, visited nodes are marked to avoid repetition, and once the exit is reached, the shortest path is highlighted with a different color in the GUI.

7. Conclusion

In conclusion, the BFS algorithm is effective for solving maze problems because it guarantees finding the shortest path from the start to the exit. The use of a graphical interface helps visualize how the algorithm explores the maze step by step, making it easier to understand. This project demonstrates the practical use of data structures such as queues and shows how algorithms can be applied to real-world problems like pathfinding.